Histogram-based Outlier Score

Assignment 2: Paper Summarization (Data Preparation)

Name: Satyam Kumar (8841438)

Paper: M. Goldstein and A. Dengel, Histogram-based Outlier Score (HBOS): A fast Unsupervised Anomaly Detection Algorithm, DFKI

---

1. Problem and motivation

Goldstein and Dengel look at how to find unusual records quickly when there are no labels and no separate clean sample to train on. Two assumptions sit underneath everything: anomalies are rare, and they look noticeably different from normal records. Speed is the practical concern. In network monitoring, for instance, large amounts of data have to be analysed almost as soon as they arrive.

According to the paper, standard neighbour search scales quadratically with the size of the dataset. Clustering can be quicker, but a model still has to be fitted, and some statistical methods are costly as well. Histograms are a cheaper way to see where the data is concentrated. Earlier security work already combined per-variable histograms, usually with fixed or hand-picked intervals. HBOS takes that idea into the unsupervised setting and adds variable bin widths for uneven distributions.

2. Method and implementation

The core idea is easy to state. A record looks unusual when its values keep falling into sparse bins. HBOS builds one histogram per feature from the data being analysed. Categorical features simply use category frequencies. Numerical features need a bin count, k. With fixed widths, the observed range is cut into k equal intervals, and each interval's height is the fraction of records that fall inside it.

With variable widths, the N values are sorted and split into consecutive groups of roughly N/k observations. Each interval runs from the smallest to the largest value in its group, and its height is the group's population divided by the interval width. So two intervals holding similar numbers of records can get different heights: the one whose values are more spread out ends up lower.

Each histogram is then divided by its own maximum, so the tallest bar is 1.0. The score is:

$$\mathrm{HBOS}(p) = \sum_{i=1}^{d} \log\left(\frac{1}{h_i(p)}\right)$$

Here, $h_i(p)$ is the normalised height for record p on feature i, across d features. Low bars push the score up, and a bar at peak height adds nothing, so a higher score means a more unusual record. This looks a lot like Naive Bayes combining independent feature probabilities, though the peak-normalised heights are not calibrated probabilities. HBOS also only ranks records; it doesn't say where the threshold should go.

3. Optimisation and special cases

Summing logarithms gives the same ranking as multiplying reciprocal heights, and the authors use it to avoid floating-point trouble when scores become extreme. Variable widths solve a separate problem. Equal-width bins can spend much of their range on empty gaps, so most records end up packed into a few intervals. For unfamiliar or long-tailed distributions the authors recommend variable widths, with the square root of N as a starting value for k.

When many records share exactly the same value, a bin may have to grow past its N/k target, and its area grows with it. The paper describes the trigger as a repetition count above k, so it isn't clear exactly when this kicks in. Boundary conventions and zero-width bins aren't covered either. These are gaps in the implementation details, though, not signs that the idea itself is flawed.

With the number of features held fixed, building and scoring cost O(N) for fixed widths and O(N log N) for variable widths. The extra cost comes from sorting. So the linear-time claim in the abstract only holds for one of the two modes.

4. Evaluation and results

The evaluation uses the UCI breast-cancer data and two pen-based benchmarks, pen-global and pen-local. Their preprocessing follows an earlier thesis and isn't explained in the paper itself. Performance is measured with AUC, using the ROC curve traced out by moving the detection threshold. HBOS is compared against ten baselines: k-NN, LOF, Fast-LOF, COF, INFLO, LoOP, LOCI, CBLOF, u-CBLOF and LDCOF. Table 1 reports results under settings called optimal, but it neither lists the values nor says how they were chosen.

Dataset	HBOS AUC	Best rival (AUC)
Breast-cancer	0.9910	INFLO (0.9922)
Pen-global	0.9214	u-CBLOF (0.9923)
Pen-local	0.7651	Fast-LOF (0.9937)

AUC of HBOS against the best competing method per dataset (from Table 1 of the paper).

HBOS comes very close to the leader on breast-cancer, drops further behind on pen-global, and does much worse on pen-local. LOCI has no pen-local result because it needed too much memory. Figure 1 shows how sensitive the methods are to k (bins for HBOS, neighbours for the rest), but the paper never says which dataset it uses.

For timing, pen-global has just 809 records, so each method was run 10,000 times on a single thread of an AMD Phenom II X6 1100T and averaged. k-NN takes 28.5 ms and LOF 28.0 ms. LDCOF uses k-means, started a single time from 30 random centroids, and needs 20.0 ms with 10 optimisation steps or 30.0 ms with the default 100. HBOS takes 3.8 ms with variable widths and 4.1 ms with fixed widths. That backs up the roughly sevenfold advantage over neighbour methods. The fivefold gain over LDCOF, however, only matches its faster configuration.

On a private dataset with 1,000,000 records and 15 features, LOF needs 23 hours and 46 minutes. HBOS needs 38 seconds, or 46 seconds with variable widths. No detection-quality results are given for this larger test.

5. Strengths and limitations

HBOS is simple, cheap to run, and comes with an implementation in a RapidMiner extension. Its weaknesses all come from ignoring how features relate to each other. Values that are common on their own can still form an odd combination, and HBOS won't give it a high score. In the same way, a point can stand out from its local neighbourhood without being rare in any single feature's distribution. That explains why local anomalies are hard for HBOS, although it doesn't prove that every local anomaly is invisible to it.

The authors credit peak normalisation with giving every feature equal weight. Rescaling doesn't fix duplicated evidence from correlated variables, though. I'd also argue the experiments support a narrower conclusion than general parity with the best global detectors. The breast-cancer result is strong, but pen-global shows a clear gap.

Reproducibility is incomplete as well. Parameter selection is described too thinly to rule out evaluation labels being used during tuning, and the AUCs come without any uncertainty estimates. Repeated timings on small datasets say little about performance in other applications, and the strongest speed comparison relies on private data nobody else can check. Overall, HBOS makes a useful fast baseline when rarity in individual features is informative, but its accuracy needs checking for each new application.

Task 2: Reproducing HBOS results

Code: https://github.com/satyamkumarcode/Infomdw-hbos-reproduction

Setup

The authors did publish their HBOS code, but only as a RapidMiner (Java) plug-in. I therefore ported the histogram and scoring classes to Python (src/hbos.py) and tested the port with 21 unit tests: hand-computed examples, invariance checks and input validation. Two small defects in the Java code are fixed on purpose. The last value of a dynamic histogram is now counted, and a value sitting exactly on a static bin border is scored in the same bin it was counted in. I also ran PyOD's HBOS as a second, independent implementation.

The competitors come from scikit-learn (k-NN, LOF), PyOD (COF, CBLOF, u-CBLOF, LOCI) and PyNomaly (LoOP). For data I used the authors' own benchmark versions from Harvard Dataverse (doi:10.7910/DVN/OPQMVF): breast-cancer (367 records, 30 features, 10 outliers), pen-global (809, 16, 90) and pen-local (6,724, 16, 10). Labels are never used for fitting, only for AUC and for the oracle parameter search.

Table 1 of the paper gives AUC at "optimal parameter settings" but doesn't say how those settings were found. So I report two numbers per method. The first is the AUC at a fixed default (HBOS: round(√N) bins; neighbour methods: k = 10; CBLOF: 8 clusters). The second is the best AUC over a sweep of k = 5–100 (2–30 clusters). Because that best value is picked with the labels, it is an optimistic oracle number. How the paper picked its own values is unknown.

Main experiment: ROC-AUC

Method	Breast-cancer			Pen-global			Pen-local		
	P	D	O (k)	P	D	O (k)	P	D	O (k)
HBOS (corrected port, dynamic)	0.9910	0.9804	0.9849 (25)	0.9214	0.7125	0.7801 (15)	0.7651	0.6618	0.7253 (50)
HBOS (corrected port, static)	0.9910	0.9829	0.9852 (5)	0.9214	0.7146	0.7661 (5)	0.7651	0.7202	0.7545 (20)
HBOS (PyOD)	0.9910	0.9560	0.9613 (5)	0.9214	0.7435	0.7518 (5)	0.7651	0.6755	0.6979 (5)
k-NN	0.9826	0.9790	0.9824 (90)	0.9892	0.9912	0.9934 (15)	0.9852	0.9862	0.9862 (10)
LOF	0.9916	0.9768	0.9840 (80)	0.8864	0.6233	0.8849 (25)	0.9878	0.9878	0.9900 (20)
COF	0.9888	0.9314	0.9644 (20)	0.9586	0.5365	0.9546 (50)	0.9688	0.9408	0.9849 (100)
LoOP	0.9882	0.9350	0.9835 (100)	0.8492	0.5426	0.8610 (50)	0.9864	0.9549	0.9892 (40)
CBLOF	0.8389	0.3420	0.7639 (6)	0.6808	0.7734	0.7734 (8)	0.7007	0.6418	0.8902 (15)
u-CBLOF	0.9743	0.9796	0.9899 (30)	0.9923	0.9237	0.9847 (15)	0.9767	0.9055	0.9444 (30)
LOCI	0.9678	0.6818	-	0.8868	0.5954	-	-	-	-

Table A. *P = paper, D = default, O = label-tuned oracle, with k or the cluster count in brackets. The paper's HBOS values are repeated in each HBOS row for comparison. LOCI is a separate run at a fixed setting. Fast-LOF, INFLO and LDCOF were not implemented, so this is only a partial reproduction.*

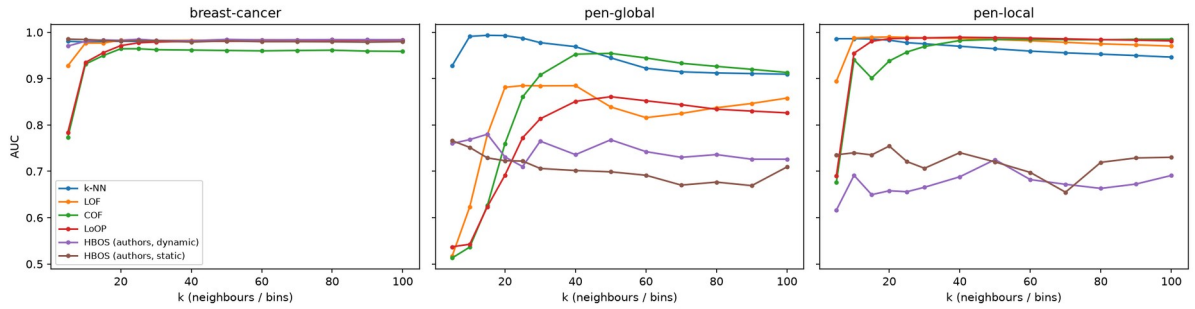


Figure A. *AUC against k on all three datasets, drawn in the style of the paper's Fig. 1 (the paper doesn't say which dataset its own figure uses).*

Comparison

Some of the tuned competitor results come very close to the paper, though not all of them. With label-tuned k, k-NN reaches 0.993 on pen-global (paper 0.989), LOF reaches 0.885 there (paper 0.886), and LOF gets 0.990 on pen-local (paper 0.988). Most neighbour-based results land within about 0.01–0.03 of the paper. That suggests the data files match the authors', but it doesn't prove it. Defaults can be much worse: LOF on pen-global drops to 0.623 at k = 10, which shows how much the paper's "optimal" settings matter. CBLOF is unstable in PyOD and misses the paper in both directions.

LOCI is the odd one out. PyOD's version gives 0.682 on breast-cancer (paper 0.968) and 0.595 on pen-global (paper 0.887). The scoring rule might explain part of this. PyOD scores a point by MDEF/σ at the last radius it checks, and it stops at the first radius where the point is flagged, instead of taking the maximum over all radii. I couldn't check how the authors' version scored radii, so this stays a possible explanation rather than a confirmed one.

For HBOS itself the picture is mixed. On breast-cancer it reaches 0.980 against the paper's 0.991, so the claim that HBOS is competitive on this global-outlier dataset holds up. Pen-global is a different story: 0.713 at the default and 0.780 at the best k, nowhere near the paper's 0.921. On pen-local I get 0.662 / 0.725 against 0.765. The paper's qualitative point, that HBOS fails on

local outliers while neighbour methods do fine, is reproduced. Its claim that HBOS is as good as the best methods on pen-global is not.

Explaining the large differences

Test	Pen-global		Pen-local	
	Default	Oracle	Default	Oracle
Corrected dynamic port	0.7125	0.7801	0.6618	0.7292
2012 dynamic (values/bin)	0.7125	0.7731	0.6618	0.7302
2012 static	0.7222	0.7341	0.7011	0.7354
Best of ten rule/mode configurations (AUC-selected)		0.7821 (Rice, dynamic)		0.7314 (Sturges, static)
Per-feature static search (oracle)		0.8873		0.9665
PyOD grid, 105 settings (oracle)		0.7620		0.7241

Table B. *The gap-grid oracles include larger k than Table A. Each bin rule is label-free on its own, but picking the best of the ten rules/modes uses label-based AUC. Neutral and optimistic AUC agree at default k, and there are no duplicate feature rows in these files. Neither observation rules out untested settings or preprocessing differences.*

None of the choices I tested gets back to the paper's pen-global AUC. Pen-global only comes close (0.887) when every feature gets its own bin count, tuned on the labels. The same search pushes pen-local up to 0.967, far above the paper's 0.765. Once parameters are set per feature, label tuning inflates apparent performance a lot, which makes the paper's undocumented selection a weak basis for comparison.

The competitors that do match don't prove the datasets are identical either. The paper only says the data were "preprocessed as in" a bachelor thesis, so an undocumented preprocessing step, or a different random sample of pen-global outliers back in 2012, is still possible. Two other explanations I couldn't test are RapidMiner's own behaviour (attribute types, handling of integer-valued features) and the vague "optimal parameter" search, which may have used settings the paper doesn't report, such as per-feature bin counts. Without the authors' exact 2012 data and process, the 0.92 can't be checked.

One caveat about my own port: it deliberately differs from the Java code at the two fixed defects. It keeps Java's next-group upper edges for dynamic bins, whereas the paper describes within-group endpoints. Comparing the source and passing unit tests doesn't guarantee that the two behave the same at runtime.

Run time

Method	Paper ms	Python ms	Relative to dynamic HBOS
HBOS (authors, dynamic)	3.8	6.53	1.00
HBOS (authors, static)	4.1	3.60	0.55
HBOS (PyOD, 10 bins)	-	5.38	0.82
k-NN	28.5	7.22	1.11
LOF	28	9.50	1.45
k-means, 30 centroids, 100 iterations	30	10.14	1.55

Table C. *Mean of 1,000 pen-global runs after warm-up, single thread, Intel64 Family 6 Model 140 Stepping 1. The paper used 10,000 repeats on an AMD Phenom II X6 1100T. Its 30.0 ms value is for LDCOF; my Python k-means skips LDCOF's scoring step, so it is only a proxy, and max_iter=100 allows early stopping.*

The absolute times can't be compared with the paper's: the CPU is different, it's Python instead of Java, and laptop timings move around with whatever else is running. The ratios are more telling. On the 809 pen-global records my HBOS is only 1.1× (dynamic) and 2.0× (static) faster than k-NN, not 7×. That's mostly an implementation effect, because my port uses plain Python loops while k-NN and LOF in scikit-learn are compiled code that picks its neighbour-search method automatically. The scaling test looks very different. On 100,000 synthetic records with 15 features, dynamic HBOS took 1.3 s and LOF took 445.8 s, and HBOS got through 1,000,000 records in 10.6 s (dynamic) and 2.3 s (static). Both Python modes sort their input, though, so these numbers don't demonstrate the paper's linear-time static construction.

Parameters

Main k grid {5,10,15,20,25,30,40,50,60,70,80,90,100}; defaults are 10 neighbours, round(√N) = 19/28/82 HBOS bins and 8 clusters. HBOS runs on raw features (dynamic and static) with peak = 1 and the sum of log10 reciprocal heights. The other methods get min-max scaling to [0,1]. k-NN uses the Euclidean distance to the kth neighbour; LOF n_neighbors = k; COF n_neighbors = k, method = fast; LoOP n_neighbors = k, extent = 3. PyOD HBOS: n_bins = k, alpha = 0.1, tol = 0.5, contamination = 0.1. CBLOF/u-CBLOF: c = {2,3,4,5,6,8,10,15,20,30}, alpha = 0.9, beta = 5, weights = True/False, contamination = 0.1, random_state = 0, with invalid cluster partitions skipped. LOCI: alpha = 0.5, k = 3.

The gap grid adds {150,200,300,400}. The per-feature grid is {2,3,4,5,7,10,15,20,25,30,40,50,60,80,100,150,200,300,400}, starting from the AUC-selected value, with at most 3 coordinate passes and a minimum improvement of 1e-12. PyOD grid: bins = {auto,5,10,20,30,50,100}, alpha = {0.01,0.05,0.1,0.2,0.5}, tol = {0.1,0.5,0.9}. Bin rules: sqrt, Sturges, Rice, Scott and Freedman-Diaconis, each in both modes. KMeans: 30 clusters, n_init = 1, max_iter = 100, seed = 0. Scaling data: standard normal, seed = 0, 15 features, N = 10,000 / 100,000 / 1,000,000, one run each; neighbour methods stop at 100,000. AUC is roc_auc_score with neutral ties = 0.5 (ties = 1 for the optimistic diagnostic). Python 3.12. Full versions, defaults, bin-rule formulas and commands are in the repository's environment/parameter files and README.